Universidad Icesi

Code, docs & tests

ResumeLens

Formal Language-Based Résumé Screening

Brayan Alejandro Jiménez Timaná · Universidad Icesi, Cali, Colombia

Computación y Estructuras Discretas III · Integrative Task 1 · 2026-2

Introduction

Résumés describe the same qualification in many ways (**JS, Javascript, JavaScript**). Keyword-based applicant tracking systems can discard qualified candidates whose wording does not match the job description. In addition, each professional profile requires a *combination* of skills, not a single keyword.

Objective

Develop **ResumeLens**, a résumé-screening tool that uses formal language models to extract, normalize and classify skills for four profiles (Full Stack, ML Engineer, DevOps, Data Analyst) and validates each candidate profile with a context-free grammar.

Literature Review

- NLP for HR treats skill taxonomies and résumé information extraction as core tasks [1].
- Most AI talent-analytics systems rely on machine learning, which is hard to explain [2].
- Finite-state transducers are a classic tool to map text variants to a normal form [3].
- **Gap:** ResumeLens makes every decision explicit and verifiable with formal models [4].

Method

Each stage is modeled with a formal model [4] and implemented in Python with `re`, `pyformlang` [5] and `textX` [6]. Skills are sorted by profile order before classification.

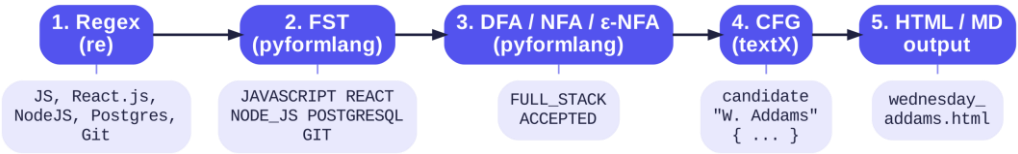


Fig. 1. ResumeLens pipeline and an example for each stage.

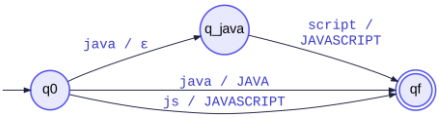


Fig. 2. Nondeterministic FST (normalization).

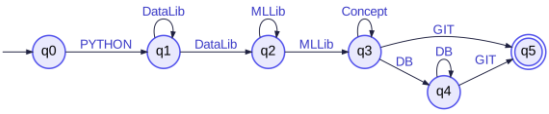


Fig. 3. NFA, ML Engineer: PYTHON DataLib MLLib* Concept* DB* GIT.*

Results

Candidate	Raw skills (sample)	Accepted profile(s)
W. Addams	JS, React.js, NodeJS, Postgres	**Full Stack (DFA)**
M. J. Watson	Python, Pandas, TensorFlow	**ML Engineer (NFA)**
B. Wayne	Linux, Docker, K8s, CI/CD	**DevOps (ε-NFA)**
L. Simpson	SQL, Power BI, Tableau	**Data Analyst (DFA)**
H. Granger	sklearn, Py Torch, PowerBI	**ML Eng. + Data Analyst**
H. Simpson	PHP, Java, Excel	**none**

111 unit tests pass

6 / 6 scenarios correct

3 automaton types

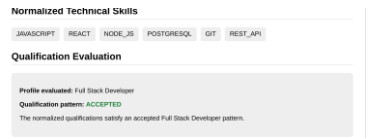


Fig. 4. HTML view of a validated profile.

Conclusions

- Each stage maps to one formal model, so every result can be traced step by step.
- Sorting skills by profile order turns each pattern into a regular language.
- The textX grammar rejects malformed profiles before any output is produced.
- **Limitations:** only listed spellings are recognized and skill level is not evaluated; future work: weighted transducers for fuzzy matching.

References

[1] N. Otani, N. Bhutani, and E. Hruschka, "Natural language processing for human resources: A survey," arXiv:2410.16498, 2024.

[2] C. Qin et al., "A comprehensive survey of artificial intelligence techniques for talent analytics," arXiv:2307.03195, 2023.

[3] M. Mohri, "Finite-state transducers in language and speech processing," *Comput. Linguist.*, vol. 23, no. 2, pp. 269–311, 1997.

[4] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Boston, MA, USA: Pearson, 2006.

[5] J. Romero, "Pyformlang: An educational library for formal language manipulation," in *Proc. 52nd ACM SIGCSE*, 2021, doi: 10.1145/3408877.3432464.

[6] I. Dejanović, R. Vaderna, G. Milosavljević, and Ž. Vuković, "TextX: A Python tool for domain-specific languages implementation," *Knowl.-Based Syst.*, vol. 115, pp. 1–4, 2017.